



BRACU ROBU · SESSION 01

Slide-by-Slide Refresher

Every slide in the deck, with the full background behind it: the physics, the numbers, the code, what to say, and the questions that land at that exact moment. Written for someone who knew all of this once and has not touched it in a while.

Read the fundamentals first, then keep this open beside the deck and stay on the same slide number. Companion to **Instructor-Handbook.pdf**, which is the run sheet, timings and general question bank.

How to use this

One entry per slide, in slide order, so you never have to search.

Part 0 is the electricity and components refresher. Read that once, cover to cover, before you open the deck. Everything after it is numbered to match the slide on screen: when the projector shows slide 22, you are on page for slide 22 here.

Each slide entry has the same shape:

- **On screen** — what the slide is doing, so you know what they are looking at.
- **What you need to know** — the real content, deeper than the slide shows.
- **Say** — a line or two that lands the idea.
- **They will ask** — the questions that come at this exact point, with answers.

P0 Fundamentals: electricity, components, breadboard	19 Four LEDs and a loop
01 Welcome	20 Button and if/else
02 Your instructor	21 Serial
03 Warm-up survey	22 The knob and the ADC
04 Roadmap	23 PWM brightness
05 Five parts of a robot	24 Servo
06 Sensing	25 Digital and analog
07 Acting	26 Logic levels
08 The loop	27 PWM in depth
09 Three kinds of computer	28 Other signal shapes
10 Inside the chip	29 Why a driver
11 16 MHz and 32 KB	30 H-bridge
12 The board family	31 Four motor types
13 Why Arduino	32 Why buses exist
14 Every part of the board	33 UART, I2C, SPI, CAN
15 Pin map	34 Which bus to pick
16 Power	35 Wrap-up
17 The workbench	RF Rapid-fire, the 20-minute version
18 Blink	

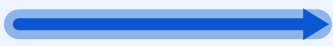
Electricity, properly

Someone will ask "but what IS voltage". Have a real answer ready.

Charge is what moves; electrons carry it. **Current (I)** is charge per second, measured in amps, and it flows *through* a component. **Voltage (V)** is energy per unit charge, the push, and it exists *between* two points.

Resistance (R) is how much a material opposes the flow.

Voltage = pressure



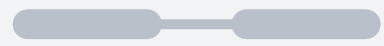
how hard it is pushed

Current = flow rate



how much passes per second

Resistance = narrowness



how much the pipe fights back

$$V = I \times R \quad \cdot \quad P = V \times I \quad \cdot \quad 1 \text{ A} = 1000 \text{ mA}$$

Power is why things get hot. A regulator dropping 12 V to 5 V while passing 500 mA is burning $(12 - 5) \times 0.5 = 3.5 \text{ W}$ as heat. Same reason a stalled motor cooks its driver.

Arrangement	Current	Voltage	Resistance
Series	Same through every part	Divides between parts	$R1 + R2 + \dots$
Parallel	Divides between branches	Same across each branch	$1/R = 1/R1 + 1/R2 + \dots$

Two consequences for class: the resistor in series with an LED carries the same current as the LED, which is how it protects it. And everything hanging off the 5 V rail is in parallel, so their currents add and the supply eventually runs out.

Ground

Ground is just the node you chose to call zero. Every voltage is quoted relative to it. Two boards that do not share that node have no agreed meaning for "3 V", which is why every guide ends with *join the grounds*, and why failing to do it produces symptoms that look exactly like a software bug.

The components on the bench

Part	Does	Number that matters	Usually got wrong by
Resistor	Limits current, divides voltage	Value in Ω , and 1/4 W rating	reading the bands from the wrong end
LED	Emits light, one direction only	Forward voltage ~2 V red, ~3.2 V blue	fitting it backwards
Capacitor	Stores charge, steadies a rail	100 nF next to a chip, 100–1000 μF near motors	reversing a polarised electrolytic
Diode	One-way valve, absorbs coil spikes	~0.7 V drop	omitting it across a relay
Transistor	Small signal switches a big current	BJT needs base current, MOSFET needs gate voltage	using a non-logic-level MOSFET on 5 V

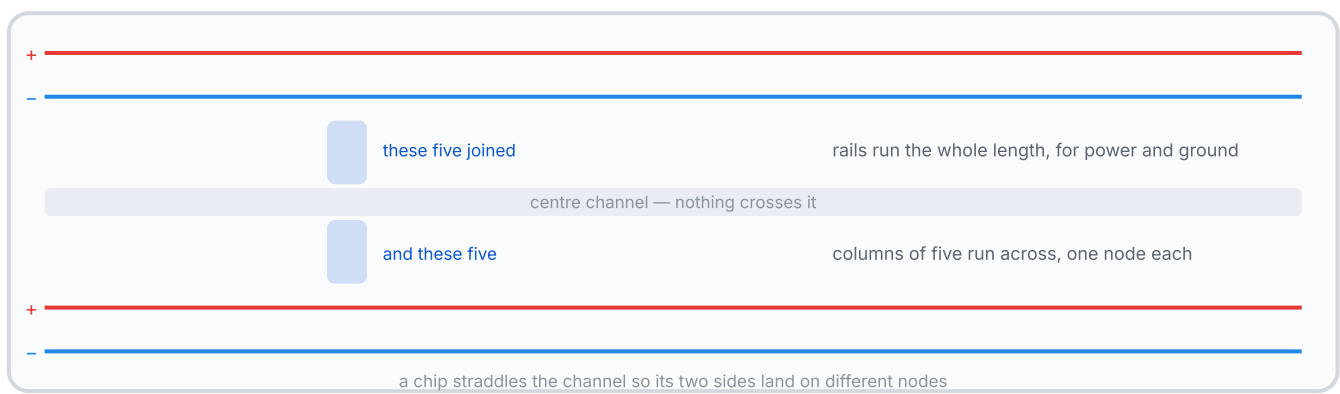
Potentiometer	Adjustable voltage divider	10 kΩ typical	wiring the wiper to 5 V instead of the pin
Pushbutton	Makes a connection	bounces 1–5 ms	rotating it 90°, so it reads permanently pressed
Buzzer	Makes sound	active = fixed tone, passive = needs tone()	trying to play notes on an active one

The LED resistor calculation, in full

$$R = (V_{\text{supply}} - V_{\text{forward}}) \div I \rightarrow (5 - 2) \div 0.015 = 200 \, \Omega \rightarrow \text{use } 220 \, \Omega$$

With 220 Ω the actual current is $(5 - 2) \div 220 = 13.6 \text{ mA}$, comfortably under the 20 mA a pin should supply. For a blue LED at 3.2 V forward, 220 Ω gives $(5 - 3.2) \div 220 = 8 \text{ mA}$, which is dimmer but perfectly safe.

The breadboard



The three breadboard faults you will fix all evening

1. Both legs of a part in the **same column**, shorting it out. Nothing lights.
2. The power rail has a break in the middle on many boards, so half the board is dead.
3. The button turned 90°, so the switched pair is the pair already joined inside. It reads permanently pressed.

01 Welcome

On screen: title, an isometric UNO with a blinking LED and a sweeping servo arm.

What you need to know: nothing technical. This slide's only job is the promise, so make the promise specific.

Say: By the end of today every one of you will have written code that made something move. Not watched me do it, written it yourself.

02 Your instructor

On screen: you with the rover arm, Mongol Tori and Tivoniq, the QR to your site.

Keep it to ninety seconds. Credibility, not biography. One sentence on what a rover has to do autonomously, one on Tivoniq as sensors in a field sending numbers to a server. Both are examples of the exact loop you are about to teach.

03 Warm-up survey

On screen: speed-round prompts, QR to the vote, a hand-count poll.

The two trick answers

Hardest for a robot: folding a towel. Stairs are solved, chess fell in 1997, face recognition is a phone feature. A towel has effectively infinite configurations, no rigid structure and no fixed grip points. This is Moravec's paradox: what humans find hard, machines find easy, and what a two-year-old does without thinking is the open research problem.

A robot arm costs less than a flagship phone: true. Hobby six-axis arms sit in mid-range phone territory. What is expensive in robotics is precision, repeatability and engineering time, not the parts.

Say: The hard problems in robotics are not the ones that look impressive. They are the ones your little sister solves without thinking about it.

04 Roadmap

Thirty seconds. Tell them the shape so nobody is wondering when the hands-on part starts.

05 Five parts of a robot

On screen: sense, think, act, power, talk, cycling automatically around a ring.

What you need to know

The classic definition is the sense-plan-act cycle. Modern practice splits "think" further into perception (what is out there), planning (what should I do) and control (make the actuators do it), but for a first session three words is the right resolution.

Power and communication are in this deck deliberately, because they are where student projects actually die. A robot that senses and decides perfectly still fails if its 5 V rail sags when the motor starts.

Ask: Is a washing machine a robot? Let them argue for a minute. It senses water level and load, decides, and acts. By this definition, yes. What it lacks is the ability to change what it does. That line is the interesting one.

They will ask: is a CNC machine or a 3D printer a robot? By this definition yes, though most people reserve the word for machines that respond to an unpredictable environment. A 3D printer follows a script; a rover reacts to a rock it has never seen.

06 Sensing

On screen: four sensors, a slider that moves the world, and the number the code sees.

The four families of sensor

Family	How it works	How you read it	Example
Switch	Makes or breaks a connection	<code>digitalRead()</code>	button, limit switch, bumper
Resistive	Resistance changes with the quantity	voltage divider into <code>analogRead()</code>	LDR, thermistor, flex sensor
Voltage output	Outputs a voltage proportional to the quantity	<code>analogRead()</code> directly	LM35 temperature, sharp IR distance
Digital / bus	Has its own chip, sends numbers	I2C, SPI or a timed pulse	MPU6050 IMU, BME280, HC-SR04

The resistive sensor trick, which they will need for a line follower

An LDR alone gives you resistance, and a pin cannot read resistance. Put it in series with a fixed resistor across 5 V and ground, and read the middle: now the changing resistance becomes a changing voltage.

$$V_{\text{out}} = 5 \text{ V} \times R_{\text{fixed}} \div (R_{\text{LDR}} + R_{\text{fixed}})$$

Pick the fixed resistor near the sensor's mid-range resistance to get the widest swing. For a typical LDR, 10 kΩ.

Ultrasonic, since it is the sensor everyone wants first

The HC-SR04 sends a 40 kHz chirp and gives you a pulse whose width is the round-trip time. Sound travels about 343 m/s at room temperature, so 29.1 μs per centimetre each way, 58.2 μs per centimetre there and back.

$$\text{distance in cm} = \text{pulse width in } \mu\text{s} \div 58$$

It fails on soft surfaces, on anything angled away, and closer than about 2 cm.

Say: Your code never sees light or heat or distance. It sees a number. Every sensor is a device for turning something physical into a number.

07 Acting

On screen: LED, buzzer, servo and DC motor, each animating when clicked.

The point to land: from the code's side, all four are the same shape. You write a number to a pin. What differs is what is on the other end of that pin and how much current it needs, which is the whole story of slides 29 to 31.

They will ask: can the Arduino run a fan / my room light / a pump? Not directly. A pin gives 20 mA. Anything with a coil or a mains plug needs a driver or a relay in between. Promise them slide 29 rather than explaining it twice.

08 The loop

On screen: a runner dot going around sense → think → act while line-follower steps light up.

This is the mental model for the whole day. A robot is not a sequence of clever ideas, it is one small loop running thousands of times a second. The intelligence lives in what the loop decides.

Worth knowing if pushed: that loop runs at some rate, and the rate matters. A line follower at 10 Hz will overshoot the line; at 500 Hz it will track it. This is the first hint of control theory, and if a sharp student asks, the words you want are "sample rate" and "the faster you close the loop the tighter you can control".

09 Three kinds of computer

On screen: laptop CPU, Raspberry Pi and microcontroller with comparison bars.

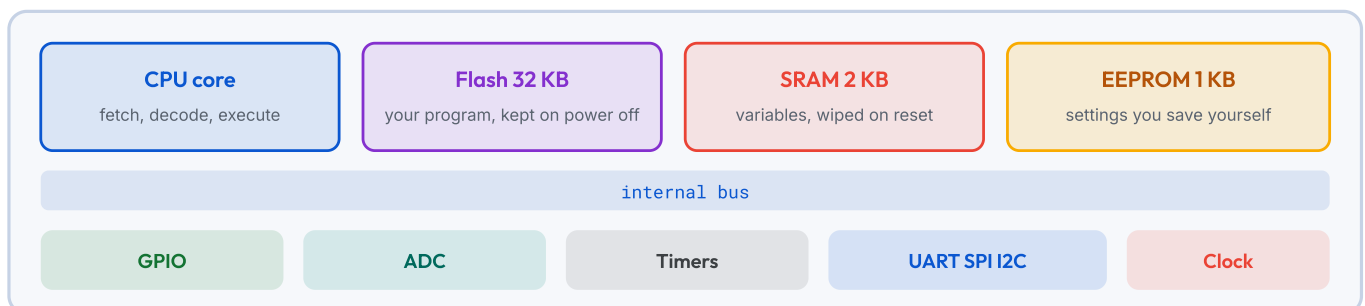
	Laptop CPU	Raspberry Pi	Microcontroller
Clock	~3 GHz, many cores	~1.5 GHz, 4 cores	16 MHz, 1 core
Memory	16 GB	2–8 GB	2 KB
Runs	an OS and anything	Linux	one program, forever
Boots in	~30 s	~20 s	under a millisecond
GPIO	none	40 pins	20 pins
Power	15–100 W	3–7 W	0.2 W
Timing	whenever the OS lets it	same problem	exact, to the microsecond

The real distinction is determinism. A laptop is fast but cannot promise *when*: the operating system may schedule something else. A microcontroller has no operating system, so if you ask it to toggle a pin every 20 μ s, it does exactly that, forever. Robotics needs that promise, which is why a rover has a Pi for vision *and* microcontrollers for the motors.

Say: Your laptop is a thousand times faster than this chip and completely useless for driving a motor, because it cannot promise to do something at an exact microsecond.

10 Inside the chip

On screen: CPU, flash, SRAM, EEPROM, GPIO, ADC and timers as hoverable blocks.



All of it on one piece of silicon a few millimetres across. That is what "microcontroller" means.

Three memories, three jobs

Memory	Size	Holds	Kept on power off	Write limit
Flash	32 KB	compiled program, minus ~0.5 KB bootloader	yes	~10,000 rewrites
SRAM	2 KB	variables, stack, buffers	no	unlimited
EEPROM	1 KB	anything you deliberately save	yes	~100,000 per cell

Harvard architecture: program and data sit in separate memories with separate buses, so the chip can fetch an instruction and touch data at the same time. The visible consequence is that string constants live in flash and must be copied into RAM to be used, which is why `F("text")` exists.

Timers are the part worth dwelling on: hardware counters that run on their own, with no code executing. You set a compare value once and the timer toggles a pin forever. That is how PWM, `millis()` and `tone()` all work.

Interrupts will come up if anyone asks how a robot notices a bumper switch while driving. A signal makes the CPU stop, run a short handler, and resume. Pins 2 and 3 on an UNO are the dedicated external interrupt pins. Rules: keep the handler tiny, no `delay()` or `Serial.print()` inside it, and mark shared variables `volatile`.

11 16 MHz and 32 KB

62.5 ns

one clock tick

~1

instruction per tick, mostly

~2 million

instructions in the time you blink

An 8-bit ALU means it works on one byte at a time. A 16-bit `int` addition takes two instructions; a 32-bit `long` takes four; a `float` multiply is a whole software routine, because there is no floating-point hardware. That is the real reason to prefer whole numbers in fast loops.

They will ask: is 2 KB really enough? Yes, because the chip is not storing anything. It holds a few dozen variables and repeats the same work forever. You run out only when you do something it was not built for, such as building long strings.

12 The board family

Board	Chip	Clock	RAM	Flash	Logic	Reach for it when
Arduino UNO	ATmega328P	16 MHz	2 KB	32 KB	5 V	teaching, and forgiving mistakes
Arduino Nano	ATmega328P	16 MHz	2 KB	32 KB	5 V	same chip, fits on a breadboard
Arduino Mega	ATmega2560	16 MHz	8 KB	256 KB	5 V	you ran out of pins
ESP32	Xtensa dual core	240 MHz	520 KB	4 MB	3.3 V	wifi, Bluetooth, real projects
Pi Pico	RP2040 dual core	133 MHz	264 KB	2 MB	3.3 V	cheap, and programmable IO blocks
STM32 Blue Pill	STM32F103	72 MHz	20 KB	64 KB	3.3 V	what industry actually ships

Be honest on this slide. The UNO is the teaching board; the ESP32 is what you would build with. Say why you are still starting here: it survives 5 V wiring mistakes, the pin numbering is simple, and every idea transfers unchanged.

13 Why Arduino

Arduino is not a language. It is C++ plus four things:

1. A **board** with the chip, crystal, regulator, USB-to-serial converter and standard headers.
2. A **bootloader** already in the chip, which runs for a moment after each reset and accepts a new sketch over plain serial. It costs about 0.5 KB of flash and replaces a hardware programmer.
3. The **core library**, which turns register writes into readable calls, and contains the `main()` you never write.
4. The **ecosystem**: IDE, library manager, and twenty years of answered questions.

```
int main(void) {
  init();           // timers, ADC prescaler, millis
  setup();          // yours, once
  for (;;) {        // yours, forever
    loop();
  }
}
```

What Upload actually does: `avr-gcc` compiles, the linker joins your code to the core and any libraries, the result becomes a `.hex` file, and `avrdude` resets the board and hands the hex to the bootloader. `Verify` does all of that except the last step, which is why it is fast.

14 Every part of the board

On screen: a drawn UNO with eleven hoverable parts.

Part	What it is doing
USB-B socket	Power in, sketch in, serial out. All three on one cable. Charge-only cables carry no data, which is the first thing to check when a board will not appear.
ATmega16U2	The second microcontroller, whose only job is to be a USB-to-serial converter. Clones replace it with a CH340, which needs a driver on some systems.
ATmega328P	The chip your code runs on. Everything else on the board exists to serve it.
16 MHz crystal	A quartz resonator giving the chip an accurate beat. Without accuracy, <code>delay(1000)</code> is not a second and serial drifts into garbage.
Voltage regulator	Takes 7–12 V from the jack down to 5 V, burning the difference as heat. Linear, so it is inefficient and cannot feed motors.
Barrel jack	For a wall adapter or battery pack when USB is not available.
Reset button	Restarts the sketch from setup. The board does this by itself at every upload.
Digital header	Pins 0–13, with ~ marking the six PWM-capable ones.
Power and analog header	3.3 V, 5 V, GND, VIN on the left; A0–A5 on the right.
TX / RX / L / ON LEDs	TX and RX flicker on serial traffic. L is wired to pin 13. ON means the rail is up.
ICSP header	For burning a bootloader onto a bare chip with a hardware programmer.

Before you click anything, ask: which chip runs your code? Most will point at the 16U2 or the crystal. That is a good moment, not a failure.

15 Pin map

Arduino pin numbers are a friendly relabelling of the chip's ports. Underneath, pins are grouped into 8-bit ports B, C and D, each with three registers.

Register	Meaning	Arduino call
DDRx	direction, 1 = output	<code>pinMode()</code>
PORTx	output value, or pull-up enable on an input	<code>digitalWrite()</code> , <code>INPUT_PULLUP</code>
PINx	the actual state of the pin	<code>digitalRead()</code>

Arduino pins	Port	Shared with
0–7	PORTD	0 and 1 are the hardware UART; 2 and 3 are the external interrupts
8–13	PORTB	10–13 are SPI; 13 drives the on-board L LED
A0–A5	PORTC	A4 is SDA and A5 is SCL, the I2C bus

The three facts that matter today: pins 0 and 1 are the serial pins so do not wire to them, ~ means PWM, and A4/A5 are also the I2C bus.

If someone asks why digitalWrite is slow: it costs roughly 50 clock cycles because it looks the pin up in a table and checks whether PWM is running on it. Writing the port register directly is one cycle. Only mention this if a student is

bit-banging a protocol.

16 Power (optional slide)

Source	Voltage	Realistic current	Notes
USB	5 V	~500 mA total	Fine for the board, LEDs and sensors.
Barrel jack	7–12 V	limited by regulator heat	Higher input voltage means more heat wasted, not more power available.
VIN pin	7–12 V	same as the jack	Same rail, just a pin.
5 V pin as an input	5 V	—	Bypasses the regulator and its protection. Only with a clean, correct 5 V.

Load	Current
One LED with 220 Ω	~15 mA
The UNO itself	~45 mA
Small servo, moving	300–700 mA
Small DC motor, running	300–1000 mA
The same motor, stalled	2000 mA and up

The brown-out story, which is the point of the slide

A servo starts moving and draws a surge. The 5 V rail dips. Below about 2.7 V the chip's brown-out detector holds it in reset. The servo stops, the rail recovers, the chip boots, the servo moves, and it all repeats. On the bench it looks exactly like a bug in the code.

The three fixes: give motors their own supply, join the grounds, and put a large capacitor across the motor supply as a local reservoir.

Regulator types, if asked: the UNO's is *linear*, which throws away the difference as heat, so 12 V in at 500 mA wastes 3.5 W. A *switching* (buck) regulator converts instead of wasting and runs at 85–95% efficiency. That is what you put on a robot with a battery.

17 The workbench

Four panels: editor with Verify and Upload, serial monitor, the drawn UNO wired to a breadboard, and the wiring list. Explain it in sixty seconds and get out. They learn by typing.

What the simulator supports: setup and loop, variables, arrays, if/else, for, while, your own functions, pinMode, digitalWrite, digitalRead, analogRead, analogWrite, delay, millis, map, constrain, tone, noTone, the Serial family, and Servo.

What it does not model: exact timing, electrical noise, brown-outs, or a component fitted backwards. Say that once, so nobody thinks a working simulation guarantees a working bench.

18 Blink

```
void setup() { pinMode(9, OUTPUT); }
void loop() {
  digitalWrite(9, HIGH);
  delay(500);
  digitalWrite(9, LOW);
  delay(500);
}
```

New idea: a pin is an output you switch between 5 V and 0 V, and `delay()` stops everything while it counts.

The electronics behind it: pin 9 goes through a 220 Ω resistor into the LED's long leg; the short leg returns to the ground rail and back to GND. The resistor sets 13.6 mA. Without it the LED tries to draw whatever it can and both it and the pin suffer.

Their change: both delays to 100.

Push further: set both to 5. The LED looks dim and steady rather than blinking. That is PWM arriving nine slides early, so name it and move on.

Why 500 and not 0.5? `delay()` takes milliseconds. For microseconds there is `delayMicroseconds()`.

What if I remove pinMode? The pin stays an input, so `digitalWrite(9, HIGH)` just switches on the internal pull-up: about 20–50 k Ω to 5 V, which gives roughly 0.2 mA. The LED may glow very faintly. A good live demo.

19 Four LEDs and a loop

New idea: an array holds four pin numbers, a `for` loop walks through them, so four LEDs cost the same code as one.

Their change: run it backwards with `for (int i = 3; i >= 0; i--)`, or add `tone(12, 1200)` for the buzzer.

Watch for `i <= 4`

An array of four has indices 0 to 3. `leds[4]` is off the end. In this simulator it is harmless; on a real chip it reads whatever byte happens to sit next in memory. C++ does not check array bounds for you, and saying that once here saves them later.

Why does tone() need a passive buzzer? `tone()` generates a square wave at the frequency you ask for. An active buzzer has its own oscillator and only knows one note, so it will click rather than play.

20 Button and if/else

New idea: `digitalRead()` plus `if / else`. Sense, decide, act, in five lines.

The pull-down, which is the real content of this slide

An input pin with nothing connected is floating: it has no defined voltage and reads noise from your hand and the mains. The button only connects the pin to 5 V when pressed, so something must hold it at 0 V the rest of the time. That is the 10 kΩ resistor to ground.

Arrangement	Idle	Pressed	Wiring
Pull-down 10 kΩ to GND	LOW	HIGH	button brings 5 V to the pin
Pull-up 10 kΩ to 5 V	HIGH	LOW	button brings GND to the pin
<code>INPUT_PULLUP</code>	HIGH	LOW	resistor is inside the chip, 20–50 kΩ, no external part

Why 10 kΩ: small enough that leakage cannot lift the pin, large enough that a press wastes only 0.5 mA.

Bounce: the contacts physically bounce for 1–5 ms, so one press can register several times. Not visible in this exercise because it only reads a level, but it bites the moment they count presses. Cure with a 20 ms delay or a millis-based filter.

Say: An input pin is not a question the chip asks the world. It is a wire, and a dangling wire will report the weather.

21 Serial

New idea: Serial is the only window into what the chip is thinking, and it goes both ways.

What baud rate actually means

Bits per second on the wire. UART has no clock line, so the receiver cannot see how fast the sender is going; it assumes the configured rate and samples in the middle of each expected bit. If the two disagree by more than roughly 2%, the sampling drifts across bit boundaries and you get garbage.

At 9600 baud with the usual 8N1 framing, each byte costs 10 bits (1 start, 8 data, 1 stop), so you get about 960 bytes per second. That is why printing a lot inside a fast loop slows the loop down: the buffer fills and `Serial.print()` starts blocking.

Why 9600? Convention only. 115200 is common now and is what an ESP32 defaults to. Any rate works as long as both sides agree.

What is the difference between '1' and 1? `'1'` in quotes is the character, ASCII 49. `1` is the number. `Serial.read()` hands you characters, so compare against `'1'`.

22 The knob and the ADC

New idea: `analogRead()` turns a voltage into a number from 0 to 1023.

How the ADC really works

One 10-bit successive-approximation converter, shared between six pins by a multiplexer, so you cannot sample two channels at the same instant. It compares the input against a reference one bit at a time, from the most significant down, like a binary search.

1024

steps across 0 to 5 V

~4.9 mV

one step, the finest change it can see

A conversion takes about 13 ADC clocks, and the ADC clock is prescaled to around 125 kHz for full accuracy, so each `analogRead()` costs roughly **100 μs**. That caps you near 9,000 reads a second and explains why reading six sensors in a tight loop is slower than expected.

The potentiometer is a voltage divider you can turn. Outer legs to 5 V and ground, wiper to A0. Turning it moves the tap, so the pin sees anything from 0 to 5 V.

What voltage is 512? About 2.5 V. Half the range.

Can I read something smaller than 5 mV? Not on the default reference. Switch to the internal 1.1 V reference with `analogReference(INTERNAL)` and one step becomes about 1 mV, over a smaller range.

23 PWM brightness

New idea: `map()` rescales one range to another; `analogWrite()` fakes an analog output.

```
map(value, fromLow, fromHigh, toLow, toHigh)
```

`map(knob, 0, 1023, 0, 255)` converts the ADC's range into PWM's range. It does integer maths, so it truncates; that is fine here and worth knowing later.

Their change: reverse it with `map(knob, 0, 1023, 255, 0)`.

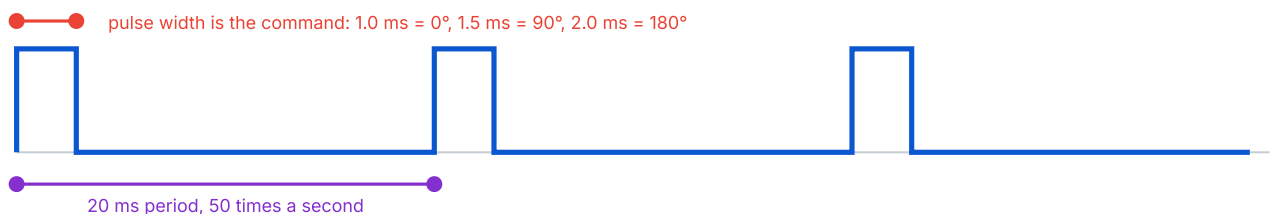
Watch for a pin without a ~

`analogWrite()` on a non-PWM pin does not fade; it behaves like `digitalWrite()`, off below 128 and on above. The workbench prints a warning rather than failing silently, which the real IDE will not do for them.

24 Servo

New idea: a library hides work someone else already debugged.

The servo protocol, which is what the library is hiding



The gap between pulses carries no information. Only the width of the high part does.

A hobby servo expects a pulse every 20 ms. The *width* of that pulse is the command, not its height and not how often it arrives. Inside the case, a potentiometer on the output shaft tells a small controller where the horn actually is; it drives the motor until the measured position matches the commanded one, and keeps correcting. That closed loop is why a servo *holds* an angle against a load, and why it buzzes when you push against it.

The Servo library uses Timer1, which is why PWM on pins 9 and 10 stops working once a servo is attached. That collision surprises everyone the first time.

Their change: hold still with `myServo.write(90);`, then drive it from the knob:

```
void loop() {  
  int knob = analogRead(A0);  
  myServo.write(map(knob, 0, 1023, 0, 180));  
}
```

Say while it sweeps: On a real bench, power that servo from its own supply and join the grounds. That one sentence saves their first project.

25 Digital and analog

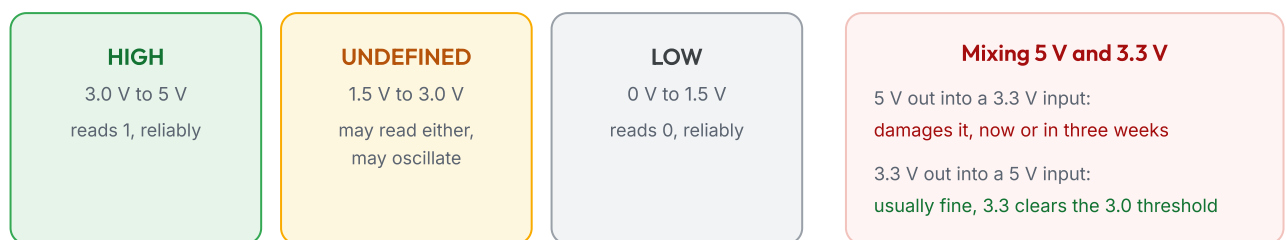
On screen: a live waveform, a source slider, and both readings side by side.

Digital means two meaningful states. **Analog** means a continuum. The real world is analog; the chip is digital; the ADC translates one way and PWM translates the other. Everything on these four slides hangs off that sentence.

Quantisation: converting analog to digital always throws information away. Ten bits gives 1024 levels, so anything finer than 4.9 mV is invisible. More bits costs money and time.

Sampling: you also take readings at moments, not continuously. If the thing you are measuring changes faster than about half your sampling rate, you get aliasing, and the reconstructed signal is wrong in a way that looks plausible. Mention the word only if a keen student pushes.

26 Logic levels



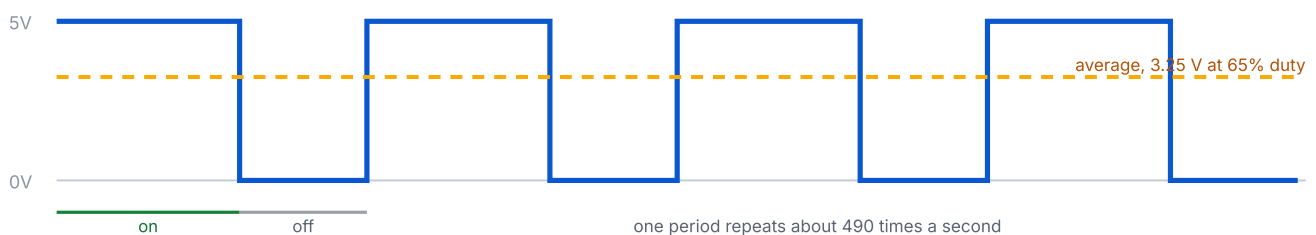
The gap between the thresholds is the noise margin. It is a design feature, not vagueness.

Shifting method	Direction	Use for
Voltage divider, 2 resistors	high to low only	slow one-way signals such as UART. Not I2C.
MOSFET shifter (BSS138)	both ways	I2C, where both ends drive the line
Buy the 3.3 V version	—	always try this first

$$V_{\text{out}} = V_{\text{in}} \times R_2 \div (R_1 + R_2) \rightarrow 5 \times 2k \div 3k = 3.33 \text{ V}$$

27 PWM in depth

The sentence that matters: the pin is never at 2.5 V. It is at 5 V half the time and 0 V the other half, and everything downstream averages it.



$$\text{duty} = \text{on time} \div \text{period} \quad \cdot \quad \text{average V} = \text{duty} \times 5 \text{ V} \quad \cdot \quad \text{analogWrite(pin, 0-255)}$$

Why it works on different loads

- **An LED** switches faster than your eye can follow. Above roughly 60 Hz you see the average brightness, not flicker. Note it is not truly linear in perceived brightness; your eye is logarithmic, so 50% duty looks brighter than half.
- **A motor** cannot start and stop 490 times a second, because it has mass. Its inertia does the averaging, and it spins at roughly the average.
- **A heater or an RC filter** averages electrically. Put a resistor and capacitor after a PWM pin and you get a real analog voltage, which is the poor man's DAC.

Frequencies on an UNO, and why they differ

Pins	Timer	Frequency	Shares with
5, 6	Timer0	about 980 Hz	<code>millis()</code> , <code>delay()</code>
9, 10	Timer1	about 490 Hz	the Servo library
3, 11	Timer2	about 490 Hz	<code>tone()</code>

Timers run independently of your code, which is the real reason PWM keeps going during a `delay()` . It also explains the collisions: attach a servo and pins 9 and 10 stop fading; call `tone()` and pins 3 and 11 stop.

Can the UNO output a real analog voltage? No, it has no DAC. PWM plus an RC filter gets you close, or use an external DAC chip over I2C. An ESP32 does have two real DAC pins.

Why does my motor not move at `analogWrite(30)`? Not enough average torque to overcome friction. Most small motors need somewhere above 20–30% duty to start, even though they will keep turning below that once moving.

28 Other signal shapes (optional slide)

Encoding	What carries the value	You meet it in
Frequency	how many pulses per second	hall sensor on a wheel, flow meters, some light sensors
Pulse width	how long each pulse stays high	servos, ultrasonic echo, RC receivers
Quadrature	two square waves a quarter cycle apart	rotary encoders on motors and knobs

Quadrature is worth understanding properly, because it is how a robot knows how far it has driven. Two sensors are offset so their outputs are 90° apart. Which one changes first tells you direction; counting the edges tells you distance. Four counts per cycle if you watch both edges of both channels, which is where "4x counting" comes from.

29 Why a driver

20 mA

a pin should supply

700 mA

a small motor running

2000 mA

the same motor stalled

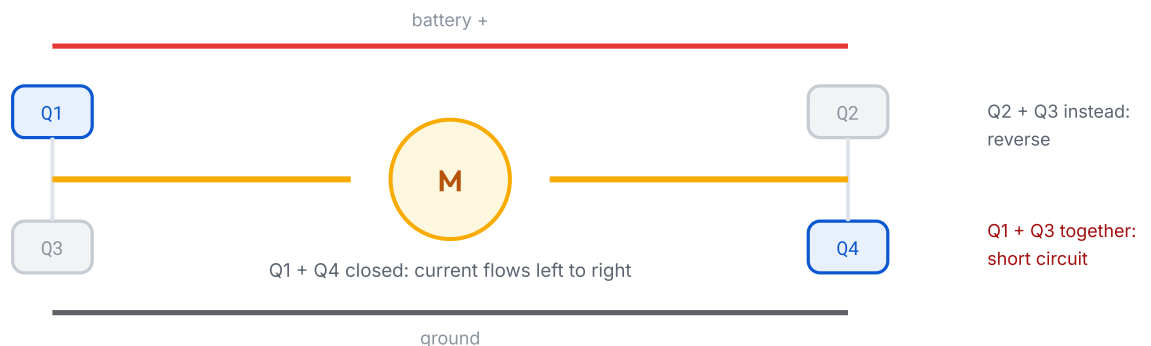
40 mA is the absolute maximum per pin before damage, and the whole chip should stay under about 200 mA total. A motor is therefore fifty times outside what a pin can do. A driver is a switch: the small signal from the pin controls it, and the large battery current flows through the driver, never through the pin.

Back-EMF, the second reason drivers exist

A motor is a coil, and a coil opposes any change in the current through it. Cut the current suddenly and the collapsing magnetic field induces a reverse voltage that can reach tens or hundreds of volts from a 6 V motor. That spike is what actually destroys pins and transistors. Drivers contain diodes that give the current a harmless path; a bare relay needs a flyback diode across its coil for exactly the same reason.

Also worth knowing: a spinning motor is a generator. Back-drive a motor by hand and it puts voltage on its terminals, which is why a robot pushed across a table can feed current back into its driver.

30 H-bridge



Four switches around the motor. Close one diagonal pair and current runs one way; close the other pair and it runs the other way. Close both switches on the *same side* and you have connected the battery straight to ground through two transistors: that is **shoot-through**, and it destroys the driver in microseconds. Real driver chips insert a deliberate dead time between switching one off and the other on, which is one reason you buy a driver rather than wiring four transistors.

Braking: closing both *low* switches shorts the motor to itself. Its own generated voltage now opposes its motion, so it stops hard. Leaving all four open lets it coast.

```
digitalWrite(IN1, HIGH); // direction
digitalWrite(IN2, LOW);
analogWrite(ENA, 180); // speed, 0 to 255
```

Two pins choose direction, one PWM pin sets speed. Every hobby driver follows this shape.

31 Four motor types

Type	How it works	Feedback	Driver	Use for
DC	Brushes commutate the coils as it spins. Speed rises with voltage, torque with current.	none	H-bridge: L298N, TB6612	wheels, fans, pumps
Servo	DC motor + gearbox + potentiometer + controller in one case. Holds a commanded angle.	built in	none, drive the signal pin	arms, grippers, steering

Stepper	Coils energised in sequence pull the rotor one fixed step at a time, typically $1.8^\circ = 200$ per turn.	none, but steps are exact	A4988, DRV8825	3D printers, CNC, axes
Brushless	No brushes; an electronic controller energises coils in sequence. High power for its weight, long life.	sensored or sensorless	ESC	drones, e-bikes, rover drive

The details that matter in practice

- **Torque and speed trade off.** A DC motor's torque is highest at zero speed (stall) and falls to zero at its top speed. A gearbox trades speed for torque: a 50:1 gearbox gives roughly 50 times the torque at 1/50 the speed.
- **Steppers lose position silently.** Miss a step through too much load or too fast an acceleration and nothing tells you. The machine is simply wrong from then on, which is why serious machines add an encoder.
- **Stepper drivers need current limiting.** There is a small trimmer on an A4988 that sets coil current. Set too high and the motor and driver cook; too low and it skips steps.
- **ESCs must be armed.** They expect a low throttle signal first, as a safety interlock. Treat a spinning propeller as a blade, because it is one.

32 Why buses exist

Four devices wired one signal per function needs about fourteen wires, and an UNO has twenty pins. A bus lets many devices share the same two or four wires and take turns. Every bus has to answer four questions: who talks first, how fast, who is being addressed, and how to tell that the data arrived intact.

33 UART, I2C, SPI and the rest

UART



8N1 framing: 10 bits carry one byte, so 9600 baud is about 960 bytes per second.

Two wires, TX and RX, crossed between the boards, plus a shared ground. No clock line, so both ends must be set to the same baud rate; more than about 2% disagreement produces garbage. Point to point only, exactly two devices.

I2C

Two wires, SDA (data) and SCL (clock), shared by up to 127 devices. The master drives the clock and starts every transaction by sending a 7-bit address; only the device with that address answers, with an ACK bit. Devices can only pull the lines *down*, never drive them up, so both lines need pull-up resistors to the supply, usually 4.7 kΩ. Most breakout boards already have them, which is why it often works with none added, and why adding four boards each with its own pull-ups eventually loads the bus too heavily.

Address clashes are the classic I2C problem: two identical sensors both answer, and the data is nonsense. Most modules have a solder jumper or an address pin to move to an alternative.

On an UNO, SDA is A4 and SCL is A5. Speed is typically 100 kHz or 400 kHz.

SPI

Four wires: MOSI, MISO, SCK and one chip select per device. Full duplex, so it sends and receives in the same clock cycles, and much faster than I2C, in the megabits. The cost is pins: every extra device needs another CS line. On an UNO: 11 MOSI, 12 MISO, 13 SCK, and 10 is the conventional CS.

If someone mentions "SPI mode", it refers to clock polarity and phase (CPOL and CPHA): whether the clock idles high or low and whether data is sampled on the rising or falling edge. Both ends must agree, and getting it wrong gives you plausible-looking garbage.

1-Wire, RS-485, CAN

Bus	Idea	Where it shines
1-Wire	Data and often power on a single wire, with timing encoded in pulse widths. Each device has a unique 64-bit ID.	DS18B20 temperature sensors; simplest possible wiring
RS-485	UART made tough: the signal is the <i>difference</i> between two wires, so noise that hits both cancels. Needs 120 Ω termination at each end of the run.	industrial sensors, Modbus, cable runs over a kilometre
CAN	Differential like RS-485, but with built-in arbitration: if two nodes start at once, the one with the lower ID wins automatically and the other retries. No master. CRC on every frame.	vehicles, and rover drive systems that must not stop when one module dies

CAN arbitration, if asked: a dominant bit physically overrides a recessive one on the wire. Every node listens while it transmits, and any node that sees a dominant bit where it sent a recessive one knows it lost and stops. No time is wasted on collisions, and the priority is built into the message ID.

34 Which bus to pick

Situation	Answer	Because
Two chips on one board, few pins spare	I2C	two wires, many devices, addresses
A screen or SD card that needs throughput	SPI	megabits, full duplex
Two boards joined by a short cable	UART	simplest possible link
One temperature sensor, one spare pin	1-Wire	a single wire will do
A sensor 200 m away in a noisy building	RS-485	differential survives the noise
A vehicle where one dead module must not stop the rest	CAN	no master, arbitration, CRC

35 Wrap-up

Read the eight recap lines back, give the homework, and close with the line about not getting stuck alone at 2 am. Then stay in the room for five minutes; the best questions of the day arrive after the official end.

If you only have the taxi ride

One line per idea, in slide order.

Slide	The one thing
05	Sense, think, act, plus power and communication. Every robot, at every scale.
06	Every sensor turns something physical into a number. Resistive sensors need a divider.
08	A robot is a small loop running thousands of times a second.
09	A microcontroller is not slow, it is <i>certain</i> . No OS means exact timing.
10	Flash holds the program, SRAM holds variables, EEPROM holds what you save. 32 KB, 2 KB, 1 KB.
11	16 MHz, 62.5 ns per tick, 8-bit ALU, no floating-point hardware.
12	UNO to learn, ESP32 to build. 5 V forgives, 3.3 V does not.
13	Arduino = board + bootloader + core library + ecosystem. Not a language.
14	The 16U2 is the USB converter; the 328P runs your code. Crystal gives accurate time.
15	0 and 1 are serial. ~ is PWM. A4 and A5 are I2C.
16	Brown-out: servo surges, rail dips below 2.7 V, chip resets, looks like a code bug.
18	Pin is 5 V or 0 V. 220 Ω sets 13.6 mA. delay blocks everything.
19	Array plus for loop. Array of 4 has indices 0 to 3.
20	A floating pin reads noise. 10 k Ω pull-down, or INPUT_PULLUP and inverted logic.
21	Baud is bits per second, no clock line, 2% mismatch gives garbage. 8N1 = 10 bits per byte.
22	10-bit ADC, 1024 steps, 4.9 mV each, about 100 μ s per read, six pins share one converter.
23	map() rescales, integer maths. analogWrite only on the six ~ pins.
24	Servo: pulse every 20 ms, width 1–2 ms is the angle. Closed loop inside. Uses Timer1, kills PWM on 9 and 10.
26	HIGH above 3.0 V, LOW below 1.5 V, undefined between. 5 V into 3.3 V kills chips.
27	Duty $\times$ 5 V = average. 490 Hz on most pins, 980 Hz on 5 and 6. Timers run without your code.
28	Frequency, pulse width, or two channels in quadrature. Same wire, different meaning.
29	20 mA per pin against 700 mA for a motor. Plus back-EMF, which is what actually kills pins.
30	Four switches, diagonal pairs. Same side together is shoot-through.
31	DC no feedback, servo closed loop, stepper exact but silent when it fails, BLDC needs an ESC.
33	UART two wires crossed. I2C two shared with addresses and pull-ups. SPI four and fast. CAN survives.

The three sentences to have ready at any moment

1. setup runs once, loop runs forever.
2. A pin is only ever 5 V or 0 V; analog output is a trick played with speed.
3. Share your grounds, and never ask a pin to drive a motor.

Where things are

Deck: so-ohan.github.io/robotics/ · keys: arrows, O for all slides, F full screen, S short version, B blank.

Run sheet and question bank: Instructor-Handbook.pdf, in the same folder.

This document: Instructor-Refresher.pdf.



BRACU Robu

BASICS OF ROBOTICS · SLIDE-BY-SLIDE REFRESHER